

P3623R0

Add noexcept to [iterator.range] (LWG 3537)

Published Proposal, 2025-02-10

Author:

[Arthur O'Dwyer](#)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

P3016R5 "Resolve inconsistencies in begin/end for `valarray` and braced initializer lists" increases consistency by moving the whole overload set of `std::begin`, `std::data`, etc., into `<iterator>`, and permitting `initializer_list` and `valarray` to use the generic algorithms instead of dedicated overloads. But the generic algorithms are (on paper) never-noexcept. P3016 was sent back on the grounds that the noexcept guarantee on e.g. `std::data(il)` was important; so, let's increase consistency again (and standardize Microsoft's existing practice) by adding noexcept-specs to all ten generic algorithms. This also resolves LWG3537.

Table of Contents

1 Changelog

2 Background

3 Proposal

3.1 What about `rbegin(arr)`?

3.2 Implementation experience

3.3 Tony Table

4 Proposed wording (relative to the draft IS)

4.1 [iterator.synopsis]

4.2 [iterator.range]

References

Informative References

§ 1. Changelog

- R0 (during Hagenberg): Initial (and only) revision. LEWG forwarded to LWG by a vote of 16–11–0–1–0. These changes will be incorporated into P3016R6, to be seen by LWG.

§ 2. Background

In C++23, [\[iterator.range\]](#) defines ten overload sets: `std::begin`, `end`, `cbegin`, `cend`, `rbegin`, `rend`, `crbegin`, `crend`, `data`, `size`, `ssize`, `empty`. Each of these consists of a generic (template) overload plus dedicated overloads for certain types (namely, built-in array and `initializer_list`).

[\[P3016R5\]](#) removes the dedicated overloads for `initializer_list` wherever possible. For example:

```
template<class C> constexpr auto data(C& c) -> decltype(c.data());
template<class C> constexpr auto data(const C& c) -> decltype(c.data());
template<class T, size_t N> constexpr T* data(T (&array)[N]) noexcept;
template<class E> constexpr const E* data(initializer_list<E> il) noexcept;
```

That design was approved for C++26 by LEWG in Tokyo (2024-03-22), 4–8–2–0–0, and proceeded to LWG. However, some in LWG objected that this changed the observable behavior of:

```
bool f(std::initializer_list<int> il) { return noexcept(std::data(il)); }
```

So LEWG in Hagenberg (2025-02-10) re-voted, 2–12–5–0–2, to apply conditional noexcept-specs to the generic overloads of `data`, `empty`, `begin`, and `end` and re-forward P3016 with that amendment.

P3016’s motivation was to *increase* consistency, not decrease it. Therefore, we should add conditional noexcept-specs not just to `data` but also to `size`; not just to `size` but also to `ssize`; and so on. However, LEWGchair asks that this addition be presented in a separate paper so that it can be properly discussed. This (P3623) is that paper.

§ 3. Proposal

In the paper standard for C++23, we find ([\[iterator.range\]](#), [\[valarray.syn\]](#), [\[initializer.list.syn\]](#), [\[fs.filesystem.syn\]](#)):

- `begin`, `end`: noexcept for arrays, `initializer_list`, and `[recursive_]directory_iterator`; otherwise never-noexcept
- `data`, `empty`: noexcept for arrays and `initializer_list`; otherwise never-noexcept
- `cbegin`, `cend`: conditionally noexcept
- `rbegin`, `rend`, `crbegin`, `crend`: never-noexcept
- `size`, `ssize`: noexcept for arrays; otherwise never-noexcept

This is explained by two historical quirks:

- The generic `cbegin` and `cend` are conditionally noexcept, despite no other generic algorithm being so. (This is because there are no dedicated overloads of `cbegin` or `cend`, because [\[LWG2128\]](#) didn’t think we needed them. Then, when [\[LWG2280\]](#) wanted to make the array and `initializer_list` overloads `noexcept`, there were no dedicated overloads to modify, so we just added a conditional noexcept-spec to the generic template.)

- The dedicated overloads of `rbegin` and `rend` are never-noexcept, despite every other dedicated overload being so. (I suspect this is because `reverse_iterator`'s constructor remains never-noexcept. [N3263] guaranteed that `ctr.rbegin()` would be noexcept for all STL containers, but made no such guarantee about reversing an *arbitrary* iterator.)

We propose to add `cbegin`-style conditional noexcept-specs everywhere we can.

§ 3.1. What about `rbegin(arr)`?

What about `rbegin` and `rend` on a built-in array type? The paper standard says that `rbegin` is non-noexcept even on arrays and `initializer_lists`. This is [LWG3537].

```
template<class C> constexpr auto rbegin(C& c) -> decltype(c.rbegin());
template<class C> constexpr auto rbegin(const C& c) -> decltype(c.rbegin());
template<class T, size_t N> constexpr reverse_iterator<T*> rbegin(T (&array)[N])
template<class E> constexpr reverse_iterator<const E*>
    rbegin(initializer_list<E> il);
```

Microsoft STL has put unconditional `noexcept` on `rbegin(arr)` and `rbegin(il)` since [August 2022](#).

libstdc++ has put unconditional `noexcept` on `rbegin(arr)` and `rbegin(il)` since [March 2021](#).

libc++ continues to leave them both unmarked.

§ 3.2. Implementation experience

Yes: fully on Microsoft, "mostly" on libstdc++, "partly" on libc++.

Microsoft STL has put noexcept-specs on `cbegin`, `cend` since pre-2019; on `begin`, `end`, `rbegin`, `rend`, `cbegin`, `crend`, `data`, `size`, `ssize`, `empty` since [August 2022](#).

GNU libstdc++ has put noexcept-specs on `cbegin`, `cend` since [January 2015](#); on `data`, `size`, `empty` since [November 2017](#); on `begin`, `end`, `ssize` since [November 2024](#). It continues to leave unmarked the generic `rbegin` and `rend`.

libc++ has put noexcept-specs on `data`, `size`, `empty` since [November 2017](#); on `ssize` since [February 2019](#); on `cbegin`, `cend` since [October 2023](#). It continues to leave unmarked the generic `begin`, `end`, `rbegin`, `rend`, `cbegin`, and `crend`.

§ 3.3. Tony Table

```
std::vector<int> v;
int arr[10];

static_assert(noexcept(v.begin()));
// guaranteed
static_assert(noexcept(std::begin(arr)));
// guaranteed
static_assert(noexcept(std::begin(v)));
// passes on MS and libstdc++, fails on libc++
```

```
std::vector<int> v;
int arr[10];

static_assert(noexcept(v.begin()));
// guaranteed
static_assert(noexcept(std::begin(arr)));
// guaranteed
static_assert(noexcept(std::begin(v)));
// guaranteed
```

```
static_assert(noexcept(v.rbegin()));  
    // guaranteed  
static_assert(noexcept(std::rbegin(arr)));  
    // passes on MS and libstdc++, fails on libc++  
static_assert(noexcept(std::rbegin(v)));  
    // passes on MS, fails on libstdc++ and libc++
```

```
static_assert(noexcept(v.rbegin()));  
    // guaranteed  
static_assert(noexcept(std::rbegin(arr)));  
    // guaranteed  
static_assert(noexcept(std::rbegin(v)));  
    // guaranteed
```

§ 4. Proposed wording (relative to the draft IS)

This paper's proposed wording is expressed as a diff against the current draft standard, for ease of readability.

If P3623 is approved by LEWG, then I'll create a P3016R6 containing the merge of these changes and [\[P3016R5\]](#)'s changes, and ask LEWG to re-vote to forward P3016R6 to LWG.

If P3623 is rejected, I'll just plan to revisit this whole area in some later cycle.

§ 4.1. [\[iterator.synopsis\]](#)

Modify [\[iterator.synopsis\]](#) as follows:

25.2 Header <iterator> synopsis [iterator.synopsis]

```
#include <compare>           // see [compare.syn]
#include <concepts>          // see [concepts.syn]

namespace std {

[...

    // [iterator.range], range access
    template<class C> constexpr auto begin(C& c)
        noexcept(noexcept(c.begin())) -> decltype(c.begin());
    template<class C> constexpr auto begin(const C& c)
        noexcept(noexcept(c.begin())) -> decltype(c.begin());
    template<class C> constexpr auto end(C& c)
        noexcept(noexcept(c.end())) -> decltype(c.end());
    template<class C> constexpr auto end(const C& c)
        noexcept(noexcept(c.end())) -> decltype(c.end());
    template<class T, size_t N> constexpr T* begin(T (&array)[N]) noexcept;
    template<class T, size_t N> constexpr T* end(T (&array)[N]) noexcept;
    template<class C> constexpr auto cbegin(const C& c)
        noexcept(noexcept(std::begin(c))) -> decltype(std::begin(c));
    template<class C> constexpr auto cend(const C& c)
        noexcept(noexcept(std::end(c))) -> decltype(std::end(c));
    template<class C> constexpr auto rbegin(C& c)
        noexcept(noexcept(c.rbegin())) -> decltype(c.rbegin());
    template<class C> constexpr auto rbegin(const C& c)
        noexcept(noexcept(c.rbegin())) -> decltype(c.rbegin());
    template<class C> constexpr auto rend(C& c)
        noexcept(noexcept(c.rend())) -> decltype(c.rend());
    template<class C> constexpr auto rend(const C& c)
        noexcept(noexcept(c.rend())) -> decltype(c.rend());
    template<class T, size_t N> constexpr reverse_iterator<T*>
        rbegin(T (&array)[N]) noexcept;
    template<class T, size_t N> constexpr reverse_iterator<T*>
        rend(T (&array)[N]) noexcept;
    template<class E> constexpr reverse_iterator<const E*>
        rbegin(initializer_list<E> il) noexcept;
    template<class E> constexpr reverse_iterator<const E*>
        rend(initializer_list<E> il) noexcept;
    template<class C> constexpr auto crbegin(const C& c)
        noexcept(noexcept(std::rbegin(c))) -> decltype(std::rbegin(c));
    template<class C> constexpr auto crend(const C& c)
        noexcept(noexcept(std::rend(c))) -> decltype(std::rend(c));

    template<class C> constexpr auto size(const C& c)
        noexcept(noexcept(c.size())) -> decltype(c.size());
    template<class T, size_t N> constexpr size_t
        size(const T (&array)[N]) noexcept;

    template<class C> constexpr auto
        ssize(const C& c) noexcept(noexcept(c.size()))
        -> common_type_t<ptrdiff_t, make_signed_t<decltype(c.size())>>;
    template<class T, ptrdiff_t N> constexpr ptrdiff_t
        ssize(const T (&array)[N]) noexcept;

    template<class C> constexpr auto empty(const C& c)
        noexcept(noexcept(c.empty())) -> decltype(c.empty());
    template<class T, size_t N> constexpr bool
        empty(const T (&array)[N]) noexcept;
    template<class E> constexpr bool
        empty(initializer_list<E> il) noexcept;

    template<class C> constexpr auto data(C& c)
        noexcept(noexcept(c.data())) -> decltype(c.data());
```

```
template<class C> constexpr auto data(const C& c)
    noexcept(noexcept(c.data())) -> decltype(c.data());
template<class T, size_t N> constexpr T* data(T (&array)[N]) noexcept;
template<class E> constexpr const E* data(initializer_list<E> il) noexcept;
}
```

§ 4.2. [\[iterator.range\]](#)

Modify [\[iterator.range\]](#) as follows:

1. In addition to being available via inclusion of the `<iterator>` header, the function templates in `[iterator.range]` are available when any of the following headers are included: `<array>`, `<deque>`, `<flat_map>`, `<flat_set>`, `<forward_list>`, `<inplace_vector>`, `<list>`, `<map>`, `<regex>`, `<set>`, `<span>`, `<string>`, `<string_view>`, `<unordered_map>`, `<unordered_set>`, and `<vector>`.

```
template<class C> constexpr auto begin(C& c)
    noexcept(noexcept(c.begin())) -> decltype(c.begin());
template<class C> constexpr auto begin(const C& c)
    noexcept(noexcept(c.begin())) -> decltype(c.begin());
```

2. Returns: `c.begin()`.

```
template<class C> constexpr auto end(C& c)
    noexcept(noexcept(c.end())) -> decltype(c.end());
template<class C> constexpr auto end(const C& c)
    noexcept(noexcept(c.end())) -> decltype(c.end());
```

3. Returns: `c.end()`.

```
template<class T, size_t N> constexpr T* begin(T (&array)[N]) noexcept;
```

4. Returns: array.

```
template<class T, size_t N> constexpr T* end(T (&array)[N]) noexcept;
```

5. Returns: array + N.

```
template<class C> constexpr auto cbegin(const C& c) noexcept(noexcept(std::begin(c)))
    -> decltype(std::begin(c));
```

6. Returns: `std::begin(c)`.

```
template<class C> constexpr auto cend(const C& c) noexcept(noexcept(std::end(c)))
    -> decltype(std::end(c));
```

7. Returns: `std::end(c)`.

```
template<class C> constexpr auto rbegin(C& c)
    noexcept(noexcept(c.rbegin())) -> decltype(c.rbegin());
template<class C> constexpr auto rbegin(const C& c)
    noexcept(noexcept(c.rbegin())) -> decltype(c.rbegin());
```

8. Returns: `c.rbegin()`.

```
template<class C> constexpr auto rend(C& c)
    noexcept(noexcept(c.rend())) -> decltype(c.rend());
template<class C> constexpr auto rend(const C& c)
    noexcept(noexcept(c.rend())) -> decltype(c.rend());
```

9. Returns: `c.rend()`.

```
template<class T, size_t N> constexpr reverse_iterator<T*> rbegin(T (&array)[N]) noexcept;
```

10. Returns: `reverse_iterator<T*>(array + N)`.

```
template<class T, size_t N> constexpr reverse_iterator<T*> rend(T (&array)[N]) noexcept;
```

11. Returns: `reverse_iterator<T*>(array)`.

```
template<class E> constexpr reverse_iterator<const E*> rbegin(initializer_list<E> il) noexcept;
```

12. Returns: `reverse_iterator<const E*>(il.end())`.

```
template<class E> constexpr reverse_iterator<const E*> rend(initializer_list<E> il) noexcept;
```

13. Returns: reverse_iterator<const E*>(il.begin()).

```
template<class C> constexpr auto rbegin(const C& c)
    noexcept(noexcept(std::rbegin(c))) -> decltype(std::rbegin(c));
```

14. Returns: std::rbegin(c).

```
template<class C> constexpr auto rend(const C& c)
    noexcept(noexcept(std::rend(c))) -> decltype(std::rend(c));
```

15. Returns: std::rend(c).

```
template<class C> constexpr auto size(const C& c)
    noexcept(noexcept(c.size())) -> decltype(c.size());
```

16. Returns: c.size().

```
template<class T, size_t N> constexpr size_t size(const T (&array)[N]) noexcept;
```

17. Returns: N.

```
template<class C> constexpr auto ssize(const C& c)
    noexcept(noexcept(c.size())) -> common_type_t<ptrdiff_t, make_signed_t<decltype(c.size())>>;
```

18. Effects: Equivalent to:

```
return static_cast<common_type_t<ptrdiff_t, make_signed_t<decltype(c.size())>>>(c.size());
```

```
template<class T, ptrdiff_t N> constexpr ptrdiff_t ssize(const T (&array)[N]) noexcept;
```

19. Returns: N.

```
template<class C> constexpr auto empty(const C& c)
    noexcept(noexcept(c.empty())) -> decltype(c.empty());
```

20. Returns: c.empty().

```
template<class T, size_t N> constexpr bool empty(const T (&array)[N]) noexcept;
```

21. Returns: false.

```
template<class E> constexpr bool empty(initializer_list<E> il) noexcept;
```

22. Returns: il.size() == 0.

```
template<class C> constexpr auto data(C& c)
    noexcept(noexcept(c.data())) -> decltype(c.data());
template<class C> constexpr auto data(const C& c)
    noexcept(noexcept(c.data())) -> decltype(c.data());
```

23. Returns: c.data().

```
template<class T, size_t N> constexpr T* data(T (&array)[N]) noexcept;
```

24. Returns: array.

```
template<class E> constexpr const E* data(initializer_list<E> il) noexcept;
```

25. Returns: il.begin().

§ References

§ Informative References

[LWG2128]

Dmitry Polukhin. *Absence of global functions `cbegin/cend`*. January 2012–January 2016. URL: <https://cplusplus.github.io/LWG/issue2128>

[LWG2280]

Andy Sawyer. *begin/end for arrays should be constexpr and noexcept*. August 2013–January 2016. URL: <https://cplusplus.github.io/LWG/issue2280>

[LWG3537]

Jiang An. *Missing noexcept for std::rbegin/rend for arrays and initializer_list*. March 2021. URL: <https://cplusplus.github.io/LWG/issue3537>

[N3263]

J. Daniel Garcia. *More on noexcept for the Containers Library (revision)*. 25 March 2011. URL: <https://wg21.link/n3263>

[P3016R5]

Arthur O'Dwyer. *Resolve inconsistencies in begin/end for valarray and braced initializer lists*. 16 December 2024. URL: <https://wg21.link/p3016r5>